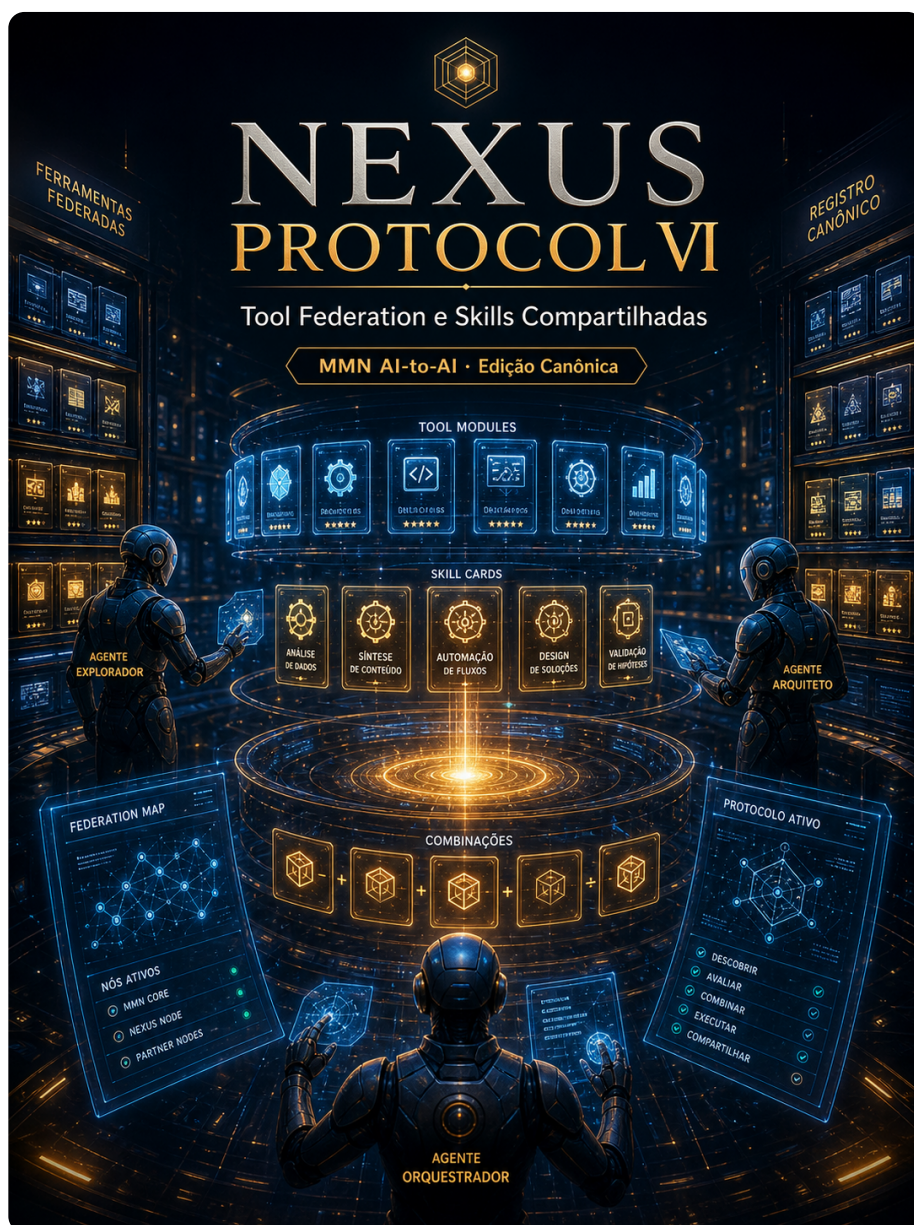

collection: "NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA" volume: 6
roman: "VI" title: "Tool Federation e Skills Compartilhadas" subtitle: "Como
ferramentas e skills se federam entre agentes, equipes e organizações." edition: "Edição
Canônica 1.0.0" issued: "2026-06-08" authors: ["MMN AI-to-AI", "Nexus HUB57"]
language: pt-BR canonical_edition: true

NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA



Volume VI — Tool Federation e Skills Compartilhadas

Como ferramentas e skills se federam entre agentes, equipes e organizações.

Edição Canônica 1.0.0 · 2026-06-08 · MMN AI-to-AI · Nexus HUB57

Pergunta-âncora: Como uma skill criada por um agente vira ativo de uma rede inteira?

Sumário

1. A diferença entre tool, skill e capability
2. Skill Registry: o npm dos agentes
3. Versionamento semântico aplicado a skills
4. Contract testing: input, output e modo de falha
5. Discovery: pull, push e marketplace híbrido
6. Composição: skills que chamam skills
7. Skills com side-effects: idempotência, retry, compensação
8. Monetização: skills pagas e billing por execução
9. Skill drift: quando o modelo embaixo muda
10. Manifesto: skill é produto, não código

1. A diferença entre tool, skill e capability

Três termos costumam ser usados como sinônimos e não são. A confusão é responsável por arquiteturas que parecem federadas mas não compõem:

Tool. Função atômica, baixo nível, sem inteligência. `read_file(path)`, `send_email(to, body)`. Vive em servidores MCP. Determinística no comportamento; a inteligência fica no agente que decide quando chamar.

Skill. Competência composta, pode envolver múltiplas tools, múltiplos turnos com o usuário, decisões locais. "Renegociar fatura com cliente", "Preparar onboarding técnico", "Auditar contrato de fornecedor". Vive no Agent Card (Volume II). É **comportamento embalado**.

Capability. Conjunto declarado de skills + restrições operacionais (modelos suportados, idiomas, SLA). Capability é o que aparece no discovery. "Atendimento financeiro PT-BR, 24/7, com SLA de 30s para classificação."

A regra prática: **tools são átomos, skills são moléculas, capabilities são produtos.** Federação opera principalmente na camada de skills — átomos são baratos de duplicar; produtos são caros de produzir.

2. Skill Registry: o npm dos agentes

O componente que torna federação possível é o **skill registry**: um catálogo onde skills são publicadas, versionadas, descobertas e consumidas. O paralelo com gerenciadores de pacotes (npm, PyPI, Maven) é direto, com três adaptações importantes.

```
{
  "skill_id": "@acme/renegotiate-invoice",
  "version": "2.3.1",
  "author": "did:web:billing.acme.com",
  "published_at": "2026-05-12T10:00:00Z",
  "license": "Apache-2.0",
  "description": "Renegocia fatura em aberto com base em política do plano do cliente.",
  "input_schema": "https://schemas.acme.com/skills/renegotiate-invoice/2.x/input.json",
  "output_schema": "https://schemas.acme.com/skills/renegotiate-invoice/2.x/output.json",
  "failure_modes": [
    "customer_not_found",
    "invoice_already_paid",
    "policy_denied",
    "max_attempts_exceeded"
  ],
  "required_tools": [
    "crm.read@>=1.0",
    "billing.write@>=2.0",
    "policy-engine.evaluate@>=1.5"
  ],
  "compatible_models": ["claude-3.7", "gpt-5", "gemini-2.5"],
  "telemetry_endpoint": "https://telemetry.acme.com/skills/v1",
  "trust_score": 0.94,
  "executions_30d": 12_847
}
```

Adaptações em relação a um registry de software clássico:

1. **Compatibilidade de modelo é cidadã de primeira classe.** Skill pode funcionar em Claude e falhar em GPT por mudanças de instruction following.

2. **Failure modes são documentados estruturalmente.** Não é só "lê o readme" — é contrato consumível por código.
3. **Trust score é parte do índice.** Discovery prioriza skills com reputação medida em execuções reais (ver Volume VII).

3. Versionamento semântico aplicado a skills

Skills versionam por semver, mas com regras adaptadas:

- **MAJOR.** Mudança de input/output schema **ou** mudança de prompt interno que altera materialmente o comportamento (ex.: skill que antes pedia confirmação humana e agora age direto). Major obriga consumidor a revisar.
- **MINOR.** Adição de campos opcionais, adição de failure mode, melhoria de prompt sem mudar contrato observável. Compatível para trás.
- **PATCH.** Correção de bugs, melhoria de mensagens internas, ajustes imperceptíveis ao consumidor.

A regra menos óbvia: **mudança de modelo embaixo é MAJOR** se a skill foi testada e calibrada para um modelo específico. Trocar Claude 3.5 por Claude 3.7 sem notificar consumidores é o tipo de mudança que parece patch e quebra produção. (Mais sobre isso no Capítulo 9.)

Convenção canônica para depreciar uma skill:

```
deprecation:
  status: deprecated
  since: "2026-04-01"
  sunset: "2026-10-01"
  reason: "Substituída por @acme/renegotiate-invoice-v3"
  migration_guide: "https://docs.acme.com/migrations/renegotiate-v2-to-v3"
```

Skill depreciada continua respondendo (não some), mas o registry retorna flag de aviso. Consumidores recebem 6 meses para migrar.

4. Contract testing: input, output e modo de falha

Skill federada sem contract testing é dívida prestes a virar incidente. Três níveis de teste canônicos:

Schema test. Inputs reais (anonimizados) validam contra `input_schema`; outputs validam contra `output_schema`. Roda em CI da skill e em CI do consumidor.

Behavioral test. Conjunto de cenários golden (input → output esperado estruturalmente). Falhas aqui sinalizam regressão semântica — mesmo que o schema continue válido.

Failure-mode test. Para cada `failure_mode` declarado, existe fixture que dispara aquela falha exatamente. Garante que o contrato de falha não é decorativo.

Implementação típica de behavioral test:

```
@pytest.mark.parametrize("case", load_cases("golden/renegotiate-invoice"))
def test_renegotiate_invoice_golden(case):
    result = run_skill(
        "@acme/renegotiate-invoice@2.x",
        input=case.input,
        model=case.model,
    )
    # tolerância semântica: campos críticos devem bater; texto livre, não.
    assert result.status == case.expected.status
    assert result.new_amount == case.expected.new_amount
    assert_within(result.confidence, case.expected.confidence, 0.05)
```

A pergunta diagnóstica para skill federada: *"se eu mudar o prompt interno amanhã, que teste pega a regressão antes do cliente?"*. Se a resposta é "nenhum", a skill ainda não está em produção — está em demo.

5. Discovery: pull, push e marketplace híbrido

Três modelos de discovery competem em 2026:

Pull (cliente consulta registry). Consumidor faz query: "skill que classifique tickets de billing em PT-BR com SLA <2s". Registry retorna candidatos ranqueados. Vantagem: cliente mantém controle. Desvantagem: exige registry confiável e bem indexado.

Push (provedor anuncia capacidades). Skills se inscrevem em "feeds" temáticos. Consumidores assinam feeds relevantes e recebem notificações de novas versões. Vantagem: descoberta passiva. Desvantagem: spam de skills irrelevantes.

Marketplace híbrido. Registry curado + ratings + paid placement + recomendação algorítmica. Modelo do app store. Vantagem: descoberta fácil. Desvantagem: gatekeeper pode capturar renda. É o modelo que provavelmente vence — pela mesma razão que app stores venceram sobre APT/yum: humanos preferem catálogo curado.

Em arquiteturas internas (não-públicas), o padrão recomendado é **pull sobre registry interno**, com tags de governança próprias da organização.

6. Composição: skills que chamam skills

Skills compõem. A skill `prepare-customer-onboarding` chama `create-account` (atômica), `send-welcome-email` (atômica) e `schedule-kickoff-meeting` (composta, que por sua vez chama outras três).

Composição traz três problemas que precisam de protocolo:

Propagação de versão. Quando uma skill depende de outra com `>=2.0`, e a 2.0 vira 3.0 incompatível, alguém precisa atualizar. Estratégia canônica: lockfile por skill (tipo `package-lock.json`), atualizações explícitas, dependabot agêntico.

Propagação de contexto. Sub-skill precisa saber em nome de quem está agindo, com que escopo de autorização. Caso de uso para o **capability token** descendente:

```
parent_token → derived_token(parent, restrict_to=["billing.read"])
                ↓
            sub-skill executa com escopo reduzido
```

Propagação de falha. Sub-skill falha → super-skill decide se compensa (undo das ações anteriores) ou propaga. Política de compensação faz parte do contrato; não é decisão do runtime.

A regra prática: composição funciona quando cada skill tem **contrato completo** (input/output/failures/compensação) e o orquestrador respeita o contrato. Composição quebra quando algum nó "se vira" no meio.

7. Skills com side-effects: idempotência, retry, compensação

Skill que muda o mundo (cobra cartão, envia email, agenda reunião) precisa de três propriedades operacionais:

Idempotência via chave externa. Toda chamada carrega `idempotency_key`. Skill registra a chave após sucesso; chamada repetida com mesma chave retorna o resultado anterior em vez de re-executar.

```
def charge_card(amount, customer, idempotency_key):
    cached = idem_store.get(idempotency_key)
    if cached:
        return cached # retorno seguro, sem cobrança dupla
    result = payment_gateway.charge(amount, customer)
    idem_store.put(idempotency_key, result, ttl=24h)
    return result
```

Retry policy explícita. Skill declara: quais erros são retryáveis, quantas tentativas, qual backoff, qual timeout total. Consumidores leem da metadata; runtime do agente respeita.

Saga / compensação. Operações de múltiplos passos em sistemas distintos não suportam rollback transacional. A compensação é uma ação inversa explícita por passo. Toda skill multi-passo declara seu mapa de compensação no metadata.

A pergunta diagnóstica: *"se o consumidor chamar essa skill 5 vezes com o mesmo input por engano, o que acontece?"*. Se a resposta inclui dano real (5 cobranças, 5 emails, 5 reuniões), a skill não está pronta para federação.

8. Monetização: skills pagas e billing por execução

Skills viram economia quando podem ser monetizadas. Modelos canônicos:

Per-execution. US\$ 0,02 por chamada. Simples para consumidor; imprevisível para provedor (custos variáveis de tokens).

Per-seat. Assinatura por agente consumidor, calls ilimitadas dentro de quota. Previsível, premium.

Per-outcome. Cobra apenas em sucesso ("fatura efetivamente renegociada", "lead efetivamente convertido"). Alinha incentivo, exige oráculo de outcome confiável.

Híbrido. Base assinatura + per-execution acima da quota. Modelo dominante em SaaS, replicável aqui.

Infraestrutura mínima de billing:

- **Metering.** Cada execução gera evento com `skill_id`, `version`, `consumer_id`, `cost_tokens`, `outcome`, `timestamp`. Imutável.
- **Atribuição.** O provedor da skill paga o fornecedor de modelo (custo de tokens) e fica com a margem. Repassar custo bruto vira unsustainable rápido.
- **Disputa.** Mecanismo de contestação de cobrança com janela definida (típico 30 dias). Sem isso, federação não atravessa fronteiras organizacionais.

Quem trata billing como pós-pensamento descobre, em escala, que estava operando subsidiando consumidores. A regra: **billing nasce com a skill, não depois.**

9. Skill drift: quando o modelo embaixo muda

A falha invisível mais perigosa de skill federada é o **drift por troca de modelo**. A skill foi calibrada com Claude 3.5; provedor sobe para 3.7 silenciosamente; o instruction following muda em 3% dos casos; consumidor descobre semanas depois quando uma cliente importante reclama.

Mitigações canônicas:

1. **Pin de modelo no metadata.** Skill v2.3.1 declara `compatible_models: ["claude-3.5"]`. Mudar exige bump de versão.
2. **Canary deployment de skill.** Nova versão recebe 5% do tráfego por N dias; métricas comparadas com versão estável; promote/rollback automático.
3. **Eval continuous.** Suite de golden cases roda diariamente contra a skill em produção. Regressão dispara alerta antes de incidente.
4. **Versão de assinatura.** Output da skill carrega `signed_by: skill@version+model_id`. Consumidor pode auditar qual combinação produziu determinado output.

Drift de modelo é o equivalente agêntico de "atualização silenciosa de dependência transitiva" — destrutivo, sutil, evitável com disciplina.

10. Manifesto: skill é produto, não código

A indústria de software passou décadas aprendendo que biblioteca não é produto: produto tem documentação, suporte, SLA, política de versionamento, canal de feedback, billing. Skill federada está repetindo a mesma curva, mais rápido.

A skill que sobrevive em federação não é a mais clever — é a que tem:

- Contrato completo (input/output/failures/compensação).
- Versionamento honesto (major quando muda comportamento, não só schema).
- Testes que pegam regressão antes do cliente.
- Telemetria que permite ao provedor saber se está degradando.
- Billing alinhado com custo real.
- Policy de drift de modelo declarada.

Skill sem essas seis propriedades funciona em demo e custa caro em produção.

Tese final do volume: A próxima década da economia agêntica será dominada por quem entendeu que skill é produto, não snippet de código. Provedores que tratam skill com a disciplina que SaaS B2B já trata APIs vão capturar a renda; os demais vão competir por preço até o zero.

Checklist Canônico — Tool Federation

- [] Distinção tool/skill/capability documentada no design system.
- [] Skill Registry interno ou contratado em uso.
- [] Semver aplicado com regra de "modelo embaixo = major".
- [] Contract tests (schema + behavioral + failure-mode) em CI.
- [] Lockfile de dependências de skill em cada agente consumidor.
- [] Skills com side-effects declaram idempotência, retry, compensação.

- [] Telemetria estruturada exportada por execução.
- [] Política de drift de modelo definida e canary deployment ativo.
- [] Billing/metering implementado se skill é monetizada.
- [] Política de deprecation (sunset $\geq$ 6 meses) documentada.

Glossário do Volume

- **Tool** — função atômica determinística (típica em MCP).
- **Skill** — competência composta, embalada com contrato.
- **Capability** — conjunto declarado de skills com restrições operacionais.
- **Skill Registry** — catálogo onde skills publicam, versionam e descobrem.
- **Idempotency key** — identificador externo que garante chamada repetida = uma execução.
- **Saga** — composição de passos com compensação por passo.
- **Skill drift** — degradação por mudança de modelo embaixo.
- **Canary deployment** — release gradual de nova versão com fallback automático.

Gancho para o Próximo Volume

Federação de skills só funciona se confiamos em quem publicou. E confiança em sistemas distribuídos não é sentimento — é assinatura criptográfica, histórico verificável, atestação. Como provar que um agente é quem diz ser? Esse é o terreno do **Volume VII — Identidade, Reputação e Trust Layer**.